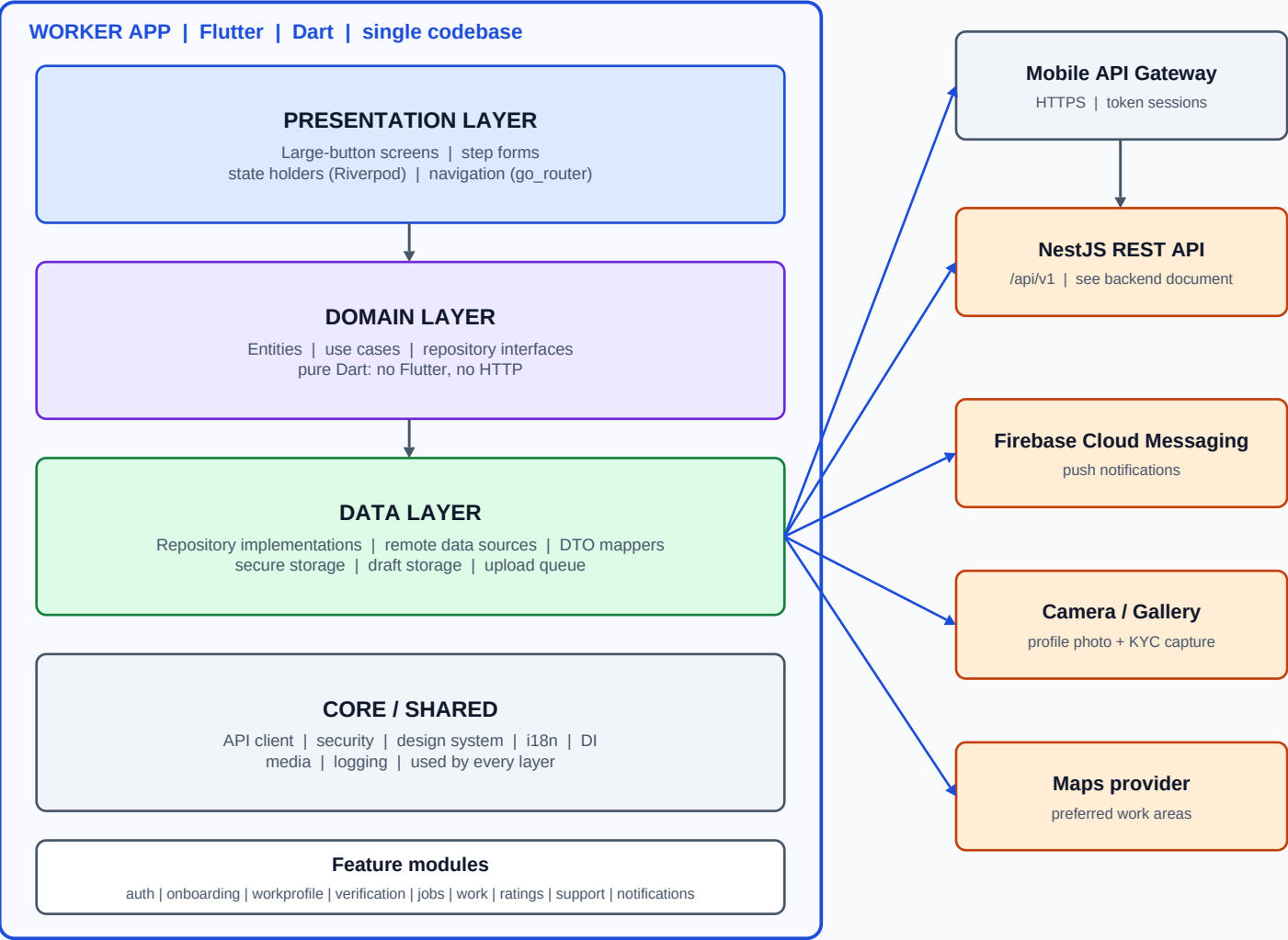


# SEVANEST — WORKER APP FRONTEND ARCHITECTURE

Flutter Architecture for the Worker Mobile Application (Android & iOS)



Calls flow downward. The Data layer implements Domain interfaces; the Domain layer never depends on Flutter or HTTP.

One Flutter codebase serves Android and iOS. Every backend call goes through the Mobile API Gateway over HTTPS with token-based sessions; only camera, maps and push are reached directly on the device.

## THE ARCHITECTURE IN SIMPLE TERMS

- **Presentation.** Large-button screens and short step forms. No API calls and no business rules here.
- **Domain.** The rules: entities and use cases such as SubmitOnboardingStep, AcceptJob, UpdateAttendance.
- **Data.** Talks to the backend, secure storage, drafts and the upload queue; turns API errors into domain failures.
- **Core / Shared.** API client, security, design system, localization, media and logging, shared with the Customer App.
- **Backend.** The server is the source of truth for verification, job and attendance status.

## DECISION STATUS

| Decision                 | Status               |
|--------------------------|----------------------|
| Flutter + Dart           | Specified by the PRD |
| Firebase FCM             | Specified by the PRD |
| Riverpod, go_router, dio | Proposed             |
| Assisted onboarding      | Proposed             |
| Telugu / Hindi UI        | Phase 2 (not V1)     |
| Numeric perf budgets     | Open item            |

ARCHITECTURAL DRIVERS

| Goal                                                     | How the architecture addresses it                                                             |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Extreme simplicity for workers with low digital literacy | Large buttons, limited text, simple status labels, one primary action per screen, short steps |
| Reliable onboarding and KYC                              | Step wizard with saved progress, resumable uploads, clear per-document status                 |
| Local-language readiness                                 | All text externalized; icon-supported labels; layouts tolerate longer text                    |
| Assisted onboarding                                      | Same wizard completed with help from operations staff or a helper (proposed assisted mode)    |
| Trust and privacy                                        | Identity documents handled securely; workers see only a requirement summary of customers      |
| Single codebase for Android and iOS                      | Flutter with platform-specific code isolated behind interfaces                                |
| Modest devices and networks (proposed assumption)        | Lean screens, compressed uploads, retry and resume, graceful failure                          |
| Maintainable by a small team                             | Layered, feature-first modules and a core library shared with the Customer App                |

**PROPOSED** Items marked Proposed are architectural choices or assumptions not fixed by the PRD. They are confirmed during the architecture phase.

**Why the stack matches the Customer App.** Flutter is specified by the PRD. The other entries on the right are proposed and match the Customer App so that code can be shared.

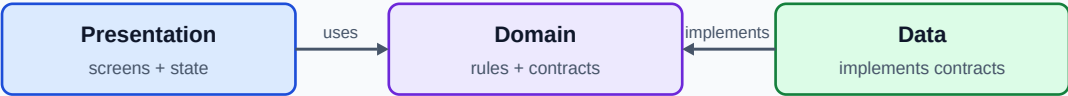
TECHNOLOGY STACK

| Area                             | Choice                                          | Notes                                                                      |
|----------------------------------|-------------------------------------------------|----------------------------------------------------------------------------|
| Framework / language             | Flutter, Dart                                   | PRD recommendation                                                         |
| State management (Proposed)      | Riverpod (alternative: Bloc)                    | Same as Customer App                                                       |
| Navigation (Proposed)            | go_router                                       | Declarative routes, auth and onboarding redirects, notification deep links |
| HTTP client (Proposed)           | dio with interceptors                           | Token handling, retry, error mapping                                       |
| Secure storage (Proposed)        | flutter_secure_storage                          | Session tokens                                                             |
| Draft and local cache (Proposed) | Lightweight local storage                       | Onboarding drafts and non-sensitive reference data                         |
| Media (Proposed)                 | Camera / gallery picker with image compression  | Profile photo and KYC capture                                              |
| Upload handling (Proposed)       | Background upload queue with retry and resume   | Reliable KYC upload on weak networks                                       |
| Push notifications               | Firebase Cloud Messaging                        | PRD recommendation                                                         |
| Maps / location (Proposed use)   | Area selection for preferred work locations     | Provider as chosen for the platform                                        |
| Localization (Proposed)          | Flutter intl with ARB files                     | Readiness for Telugu, Hindi and others in Phase 2                          |
| Crash and analytics (Proposed)   | Firebase Crashlytics and analytics events       | Confirm tool choice                                                        |
| Testing                          | flutter_test, mocktail, integration_test        | Plus usability testing with workers                                        |
| CI/CD                            | Pipeline for build, test, signing, store upload | Environments defined by DevOps engineer                                    |

LAYER RESPONSIBILITIES

| Layer         | Responsibility                                                                                                | Rules                                                            |
|---------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Presentation  | Large-button screens, step forms, status labels, navigation events and state holders                          | No API calls or business rules                                   |
| Domain        | Entities, use cases (for example SubmitOnboardingStep, AcceptJob, UpdateAttendance) and repository interfaces | Pure Dart                                                        |
| Data          | Repository implementations, remote data sources, DTOs, mappers, secure storage, draft storage, upload queue   | Implements domain interfaces; maps API errors to domain failures |
| Core / Shared | API client, security, design system, localization, media, error and logging utilities                         | No feature-specific logic                                        |

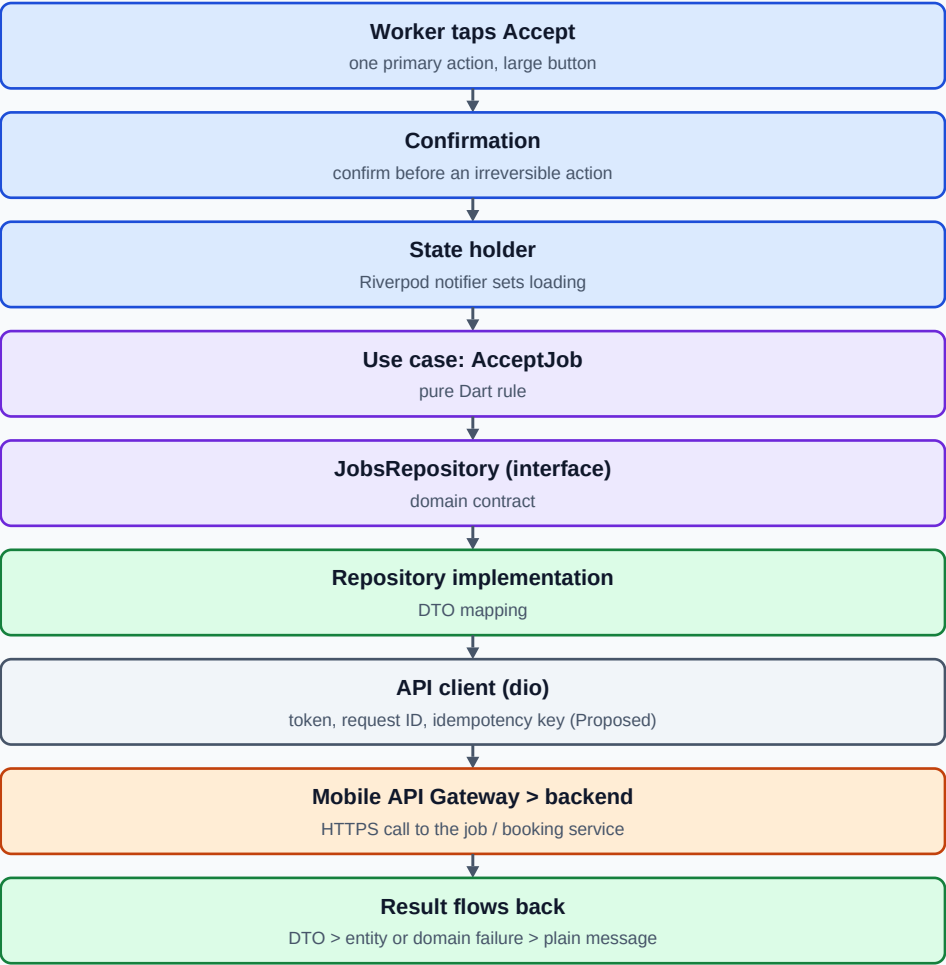
DEPENDENCY RULE



COMMUNICATION WITH THE BACKEND

- All calls go through the Mobile API Gateway over HTTPS with token-based sessions.
- Each feature talks to its matching backend service area: Authentication, User/Profile, Worker Verification, Service/Category and Availability, Matching/Search, Booking, Notification and Support/Complaint.
- Only on-device services (camera, maps, FCM) are accessed directly.

EXAMPLE: ACCEPT A JOB - HOW ONE TAP TRAVELS



**Error handling.** Failures return as domain errors (network, unauthorized, validation, not found, conflict, server, unknown) and are shown as one short sentence with one next step.

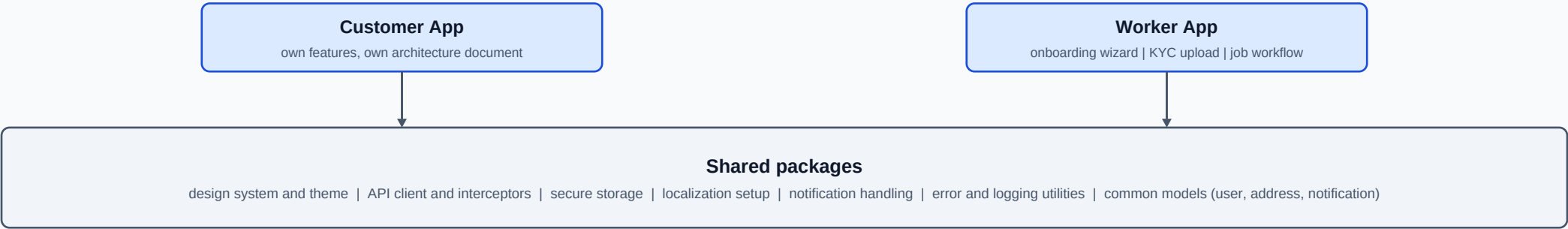
FEATURE-FIRST FOLDER LAYOUT (PROPOSED)

```
worker_app/  
  lib/  
    main_dev.dart, main_stage.dart, main_prod.dart (flavors)  
    app/          app widget, router, theme wiring, bootstrap  
    core/         network, security, storage, media, upload queue,  
                  error, logging  
    shared/       design system widgets, status labels, l10n  
    features/  
      auth/       OTP login, session  
      onboarding/ step wizard, drafts, assisted mode  
      workprofile/ roles, experience, languages, areas,  
                  salary, timings  
      verification/ document status, resubmission  
      jobs/       opportunities, requirement summary,  
                  accept / decline  
      work/       schedule, confirmation, attendance,  
                  earnings reference  
      ratings/    ratings received  
      support/    requests and complaints  
      notifications/ push handling, inbox, deep links  
    features/<name>/{presentation, domain, data}  
  test/  integration_test/
```

FEATURE MODULES AND MAIN SCREENS

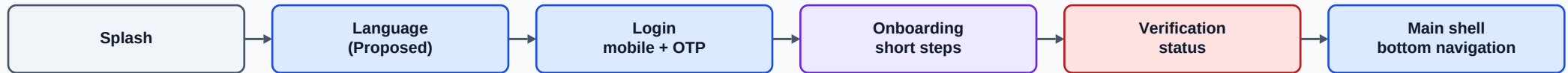
| Feature module          | Main screens                                                                                                                |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| auth                    | Splash, Login, OTP entry                                                                                                    |
| onboarding, workprofile | Photo, KYC documents, address and emergency contact, previous employer, role and experience, areas, salary and timings      |
| verification            | Verification status, resubmit document                                                                                      |
| jobs                    | Opportunity list, Requirement summary, Accept / Decline                                                                     |
| work                    | Interview / trial schedule, Job confirmation and start date, Current job, Attendance / status, Earnings / payment reference |
| ratings                 | Ratings received                                                                                                            |
| support                 | Create request / complaint, Request status                                                                                  |
| notifications           | Notification inbox, push deep links                                                                                         |

CODE SHARING WITH THE CUSTOMER APP (PROPOSED)

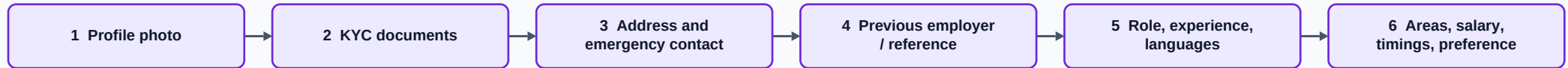


**Rule.** Worker-specific code (onboarding wizard, KYC upload, job workflow) stays inside the Worker App. The Worker theme applies larger sizing tokens on top of the shared design system, so both apps look related but the Worker App is easier to read and tap.

## APP FLOW



## ONBOARDING STEPS - progress saved after each step

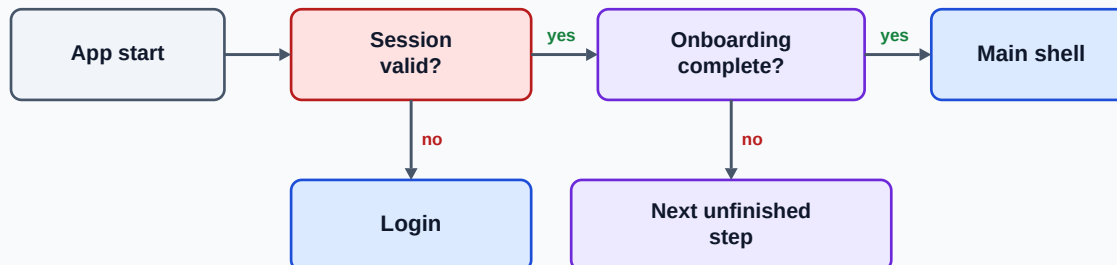


## MAIN SHELL TABS - shown after verification status



Push notifications deep-link to Jobs, My Work and Support. Large buttons, limited text and simple status labels throughout.

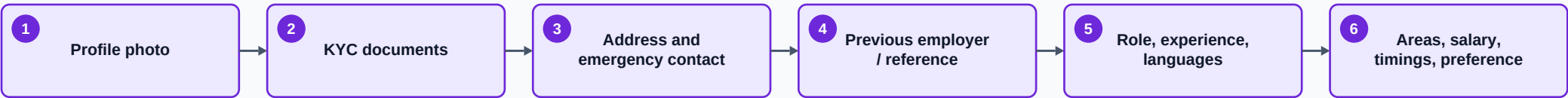
## REDIRECT RULES



## NAVIGATION RULES

- Router is declarative; routes are named and typed.
- Not logged in goes to Login; logged in with incomplete onboarding goes to the next unfinished step; otherwise to the main shell.
- Main shell has few tabs (Jobs, My Work, Support, Profile) so every function is one tap away.
- A visible back option and a plain home action exist on every screen.
- Push notifications carry a route and identifier so the app opens the right job, schedule or support request.
- Verification status is reachable from Profile and shown prominently until approval.

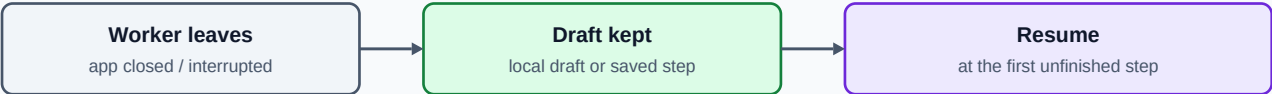
STEP WIZARD - SHORT STEPS, ONE PURPOSE EACH, ONE LARGE PRIMARY ACTION



Progress indicator shows the current step and how many remain (example: step 2 of 6)



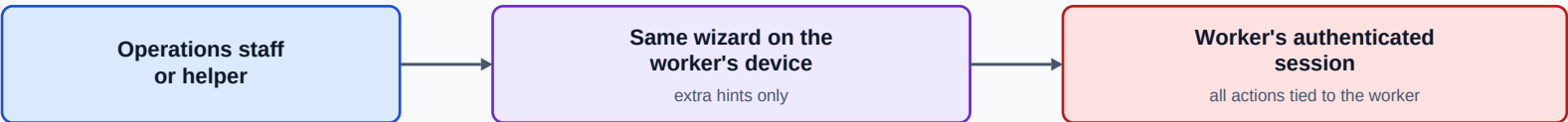
LIFECYCLE OF EVERY STEP



Draft data is stored locally until the step is submitted, then discarded. Each step validates locally before saving.

ASSISTED ONBOARDING (PROPOSED)

PROPOSED



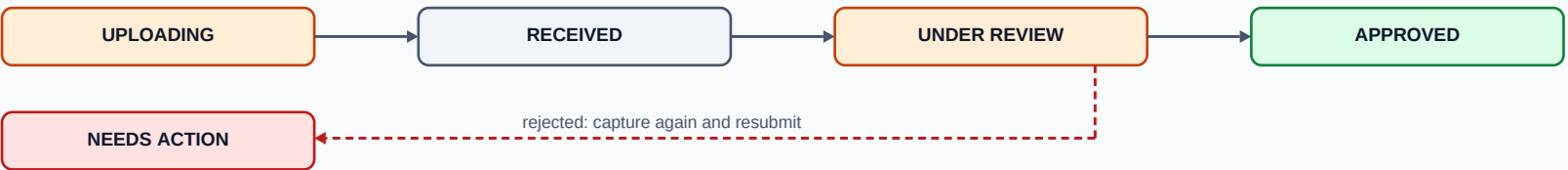
- Changes presentation only (extra hints), never data rules.
- Staff entry through the admin portal is handled on the admin side.

**Why this design.** Low digital literacy is a stated adoption risk (PRD section 20). The wizard, saved progress and assisted mode exist so a worker can finish registration without needing to read long forms or finish in one sitting.

CAPTURE AND UPLOAD PIPELINE



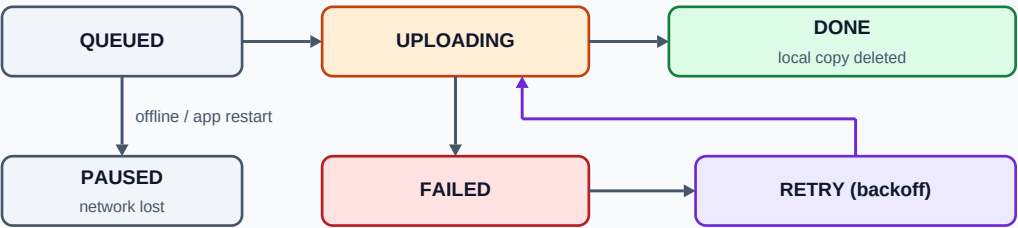
PER-DOCUMENT STATUS



The app shows only one plain status per document. Backend verification states (see backend document) are translated to these labels by a single mapping class.

UPLOAD QUEUE BEHAVIOUR

PROPOSED



On app restart or when the network returns, paused uploads resume. Backend file type and size rules are enforced on the device and again on the server.

PRIVACY AND SECURITY RULES

- Documents are sent over HTTPS to restricted storage; the app never stores them in a public or shared location.
- Local copies are deleted after a successful upload and when the worker logs out.
- The app does not display full identity documents back beyond the preview taken at capture, and shows only masked identifiers in routine screens.
- Camera and storage permissions are requested only when needed, with a short explanation and a clear path if denied.

**Same rule on both sides.** Aligned with the backend: KYC files land in private object storage, are viewed only through role-restricted signed access, and every access is audited (see Backend Architecture, page 9).

DESIGN RULES (VALUES SUBJECT TO DESIGN REVIEW)

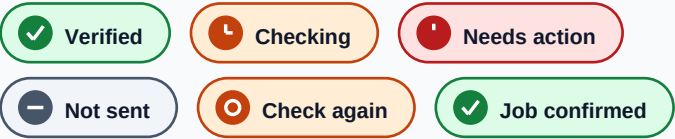
| Rule                | Standard (Proposed)                                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------------------------------|
| Large buttons       | Touch targets at least the platform minimum (about 48 dp) and larger for primary actions; full-width primary button |
| Limited text        | Short sentences; one instruction per screen; details behind a clearly labelled option                               |
| Icons with labels   | Every icon has a text label; icons support recognition and later translation                                        |
| One primary action  | Each screen has one dominant action                                                                                 |
| Readable type       | Larger base text size than the Customer App; respects system font scaling                                           |
| Contrast and colour | Status never relies on colour alone: text and icon accompany colour                                                 |
| Short forms         | A maximum of a few fields per step; numeric and selection inputs preferred over free text                           |
| Forgiving input     | Saved progress, easy edit, confirmation before irreversible actions such as decline or submit                       |

STATUS LABEL MAPPING (PROPOSED)

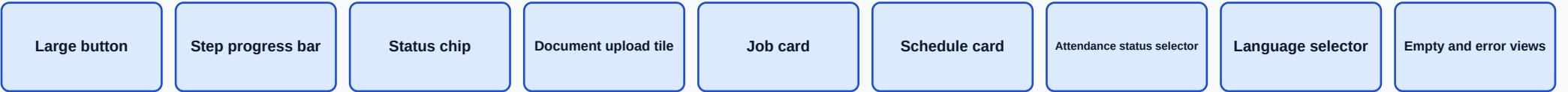
| Area         | Backend status (examples)             | Worker label               |
|--------------|---------------------------------------|----------------------------|
| Verification | Not submitted / Submitted / In review | Not sent / Sent / Checking |
| Verification | Approved                              | Verified                   |
| Verification | Rejected                              | Needs action               |
| Verification | Recheck due                           | Check again                |
| Job          | Matched                               | New job                    |
| Job          | Interview / Trial scheduled           | Interview set              |
| Job          | Confirmed                             | Job confirmed              |
| Job          | Active                                | Working                    |
| Job          | Completed / Cancelled                 | Finished / Cancelled       |

One mapping class converts a backend status into a short label, an icon and a next action. Unknown statuses fall back to a generic label instead of failing.

HOW LABELS LOOK ON SCREEN



REUSABLE COMPONENTS (STATELESS, DRIVEN BY DATA)



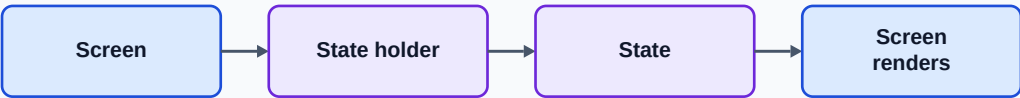
**Why stateless.** Stateless components keep the UI testable and consistent with the Customer App design system. They implement the PRD usability requirement: large buttons, limited text, simple status labels, local-language readiness and complex forms split into short steps.



STATE BY FEATURE (RIVERPOD, PROPOSED)

| Feature          | State held                                                        | Lifetime        |
|------------------|-------------------------------------------------------------------|-----------------|
| Session (global) | Authentication status, user identity, language                    | App lifetime    |
| Onboarding       | Current step, draft data, upload progress                         | Until submitted |
| Work profile     | Roles, experience, languages, areas, salary, timings, preference  | Feature         |
| Verification     | Per-check status, remarks, resubmission state                     | Feature         |
| Jobs             | Opportunity list, selected opportunity, accept / decline progress | Feature         |
| Work             | Current job, schedule, attendance entries, earnings reference     | Feature         |
| Support          | Request list and draft                                            | Feature         |
| Notifications    | Unread count, inbox                                               | App lifetime    |

HOW A SCREEN RENDERS

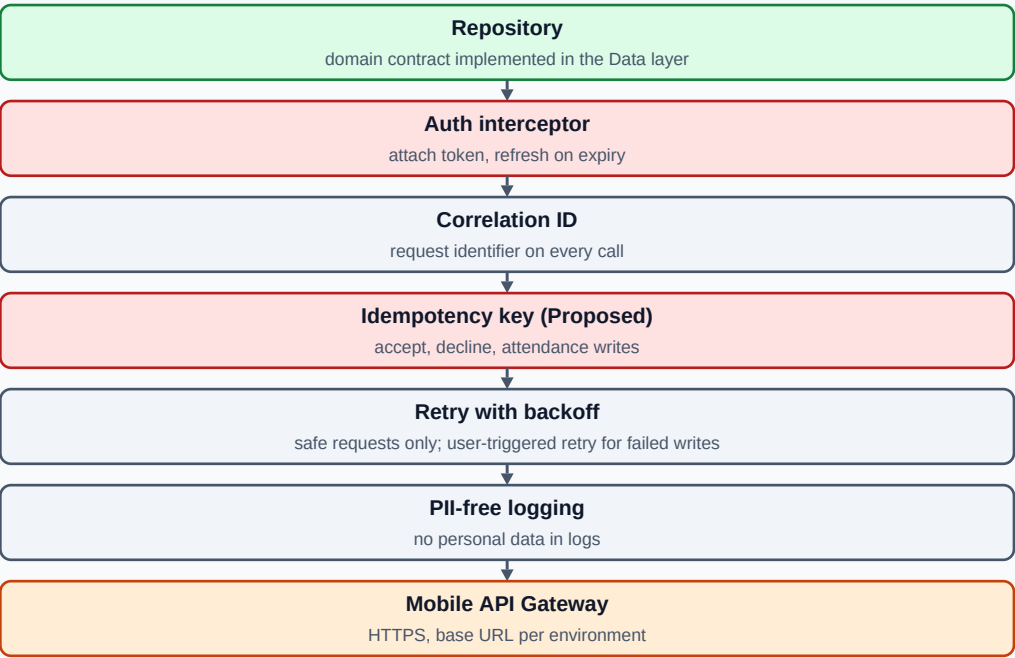


Every async screen has:



- One state holder per screen or flow, scoped to the feature and disposed with it.
- Immutable state; screens render from state.
- The server is the source of truth for verification, job and attendance status.

API CLIENT - EVERY CALL PASSES THE SAME PIPELINE



FAILURES MAP TO DOMAIN ERRORS



Repository interfaces live in the Domain layer; implementations live in the Data layer. API responses are mapped from DTOs to domain entities, so screens never see raw API shapes.

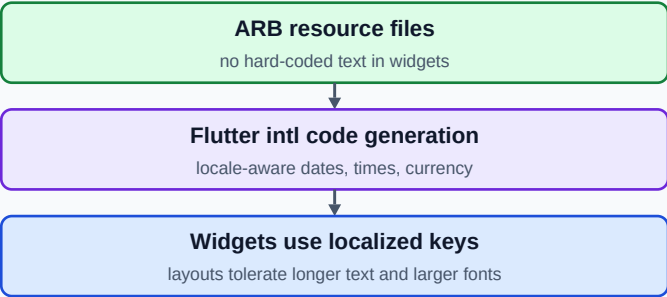
**Why idempotency.** Idempotency keys make repeated taps safe: a worker who taps Accept twice on a weak network produces one accepted job, matching the backend's Idempotency-Key handling.

POOR NETWORK BEHAVIOUR (PROPOSED)

| Data / action                            | Behaviour                                                                  |
|------------------------------------------|----------------------------------------------------------------------------|
| Onboarding steps                         | Drafts saved locally; continue when online                                 |
| Document uploads                         | Queued with retry and resume                                               |
| Opportunities and jobs                   | Show last loaded data with a "last updated" indicator; refresh when online |
| Accept / decline / attendance            | Require network; clear message and retry; never silently dropped           |
| Reference data (roles, areas, languages) | Cached locally and refreshed on launch                                     |
| Session token                            | Secure storage until logout or expiry                                      |

**Scope.** V1 requires graceful failure on weak networks; a full offline mode is not required.

LOCALIZATION READINESS



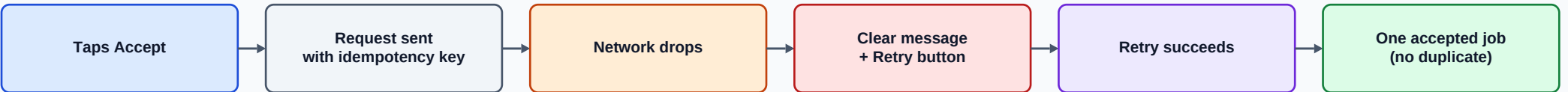
- Language selection step (Proposed) at first launch; changeable from Profile. The choice is also sent to the backend for notification language.
- V1 ships with languages approved by the business; Telugu, Hindi and others are part of the Phase 2 multi-language UI plan.
- Icons and short labels reduce reliance on reading; audio / voice help is a possible later enhancement, not V1.

PERFORMANCE APPROACH

| Area                      | Approach                                                                                          |
|---------------------------|---------------------------------------------------------------------------------------------------|
| Launch and common screens | Lazy-load features; defer non-essential initialization; stay responsive on normal mobile networks |
| Lists                     | Pagination and lazy builders for opportunities                                                    |
| Images                    | Compress before upload; cached thumbnails                                                         |
| Network                   | Cancel stale requests; avoid chatty calls; retry with backoff                                     |
| Device range (Proposed)   | Test on low-end devices and small screens; avoid heavy animations                                 |
| Memory                    | Dispose controllers and queues with their features                                                |

**Open item.** The PRD sets qualitative targets only. Numeric performance budgets are an open item to be agreed before performance testing.

WHAT A WORKER SEES ON A WEAK NETWORK

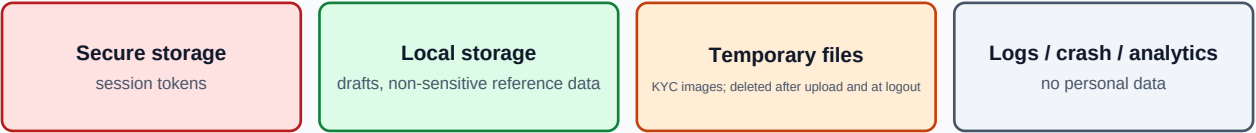


**Principle.** Writes are never dropped silently: the worker always sees either success or a plain message with a next step. Reads fall back to the last loaded data with a "last updated" indicator.

SECURITY ARCHITECTURE

| Concern               | Approach                                                                                                                               |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Transport             | HTTPS for all traffic; certificate validation enforced; certificate pinning to be evaluated (Proposed)                                 |
| Authentication        | Mobile OTP then token-based session; tokens only in secure storage                                                                     |
| OTP abuse             | App respects resend timers and attempt limits and shows backend rate-limit messages                                                    |
| KYC and personal data | Uploaded over HTTPS to restricted storage; local copies removed after upload; masked identifiers in routine screens                    |
| Customer data         | Worker sees a requirement summary only, following the privacy policy                                                                   |
| Local data            | No sensitive personal data in plain local storage, logs, crash reports or analytics events                                             |
| Secrets               | No API secrets in the app bundle; environment configuration injected at build time                                                     |
| Account state         | Suspended or blacklisted workers are signed out or restricted and shown a clear message                                                |
| Release hardening     | Code obfuscation and debug-symbol upload for release builds (Proposed)                                                                 |
| Privacy and consent   | Terms, privacy policy and consent flows shown per legally reviewed content before production release; retention rules are an open item |

WHERE DATA LIVES ON THE DEVICE



INTEGRATIONS

| Integration        | Frontend approach                                                                                                                                                                    |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Push (FCM)         | Token registration after login and refresh on change; foreground, background and cold-start handling through the notifications module, with deep links to jobs, schedule and support |
| Camera and gallery | Behind a MediaService interface with permission handling and compression                                                                                                             |
| Maps / location    | Used to select preferred work areas; manual selection from configured areas is the fallback if permission is denied                                                                  |
| SMS OTP            | Delivered by the backend; automatic OTP reading is optional (Proposed)                                                                                                               |
| Payments           | The Worker App has no payment flow. Earnings / payment reference is shown where applicable; payroll is out of scope for V1                                                           |

OBSERVABILITY AND ERROR HANDLING

- Crash reporting and non-fatal error capture in production, with release symbols.
- Structured logging without personal data.
- Analytics events (Proposed): onboarding step started / completed / abandoned, document upload success / failure, verification outcome viewed, opportunity viewed / accepted / declined, support request.
- Error messages are short, plain-language and give one next step; technical detail goes to logs only.

TESTING STRATEGY

| Level          | Scope                                                                                                                    | Tooling                           |
|----------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Unit           | Use cases, repositories, mappers, validators, status mapping, state holders                                              | flutter_test, mocktail            |
| Widget         | Design system components and key screens in all states and font scales                                                   | flutter_test                      |
| Integration    | Login, onboarding with interruption and resume, document upload with network loss, accept / decline, attendance, support | integration_test                  |
| Device and OS  | Agreed Android and iOS device matrix including low-end devices                                                           | Device farm or physical devices   |
| Usability      | Trial with representative workers and assisted-onboarding staff to validate simplicity                                   | Observation and feedback sessions |
| Non-functional | Performance, security and accessibility basics                                                                           | Profiling and static analysis     |
| UAT            | Critical workflows per acceptance criteria on agreed devices                                                             | Manual with business team         |

ENVIRONMENTS AND FLAVORS

| Flavor | Backend     | Purpose           |
|--------|-------------|-------------------|
| dev    | Development | Daily development |
| stage  | Staging     | QA and UAT        |
| prod   | Production  | Store release     |

Environment configuration is injected at build time; no secrets live in the app bundle.

CI/CD AND RELEASE



Semantic versioning with build numbers and a changelog. A minimum-supported-version check lets the backend prompt for updates (Proposed). Production monitoring and crash alerts are active at release.

ARCHITECTURE SUMMARY

|                    |                                                                                  |
|--------------------|----------------------------------------------------------------------------------|
| Framework          | Flutter / Dart, one codebase for Android and iOS (PRD)                           |
| Architecture       | Layered clean architecture, feature-first modules, core shared with Customer App |
| State / navigation | Riverpod   go_router with auth and onboarding redirects (Proposed)               |
| Network            | dio via Mobile API Gateway; HTTPS + token-based sessions                         |
| Storage            | Secure storage for tokens; local drafts; no sensitive data in plain storage      |
| Uploads            | Background queue with compression, retry and resume                              |
| UI standard        | Large buttons, limited text, simple status labels, short steps                   |
| Localization       | ARB / intl readiness; Telugu and Hindi UI in Phase 2                             |

OPEN OR PROPOSED ITEMS

- Numeric performance budgets
- Certificate pinning evaluation
- Automatic OTP reading (optional)
- V1 language list approved by the business
- Data retention rules
- Crash / analytics tool choice
- Agreed device and OS matrix

The SevaNest Worker App is designed as a simple, layered and feature-first Flutter application: workers with low digital literacy can register, verify, receive jobs and manage their work through short steps, reliable uploads and plain status labels, while the backend remains the source of truth.